



資料結構

Data Structure

Lab10

姓名：李品融

學號：113310217

Lab10-Q1

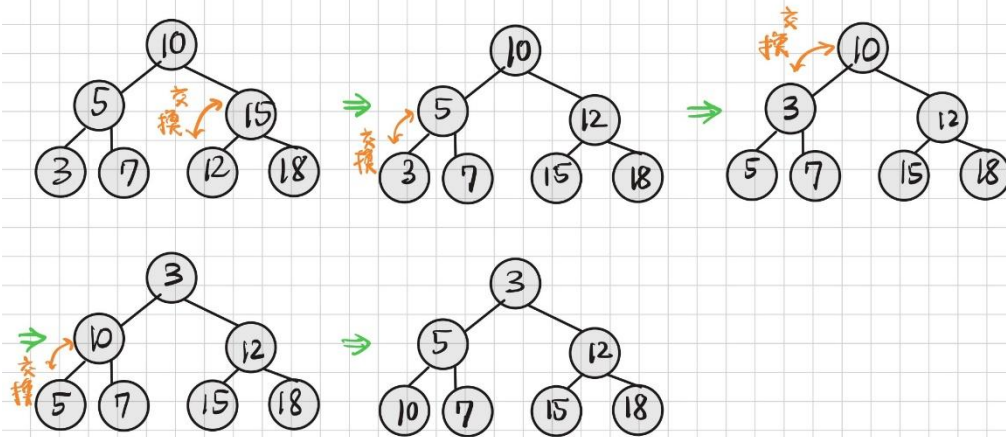
這題要求我們將提供的三個 input.txt 檔案中的元素，手繪出 Max Heap 和 Min Heap，並將詳細的步驟畫出來，使我們更了解 Max Heap 與 Min Heap 的細部原理。

Result

Input 1: 10,5,15,3,7,12,18

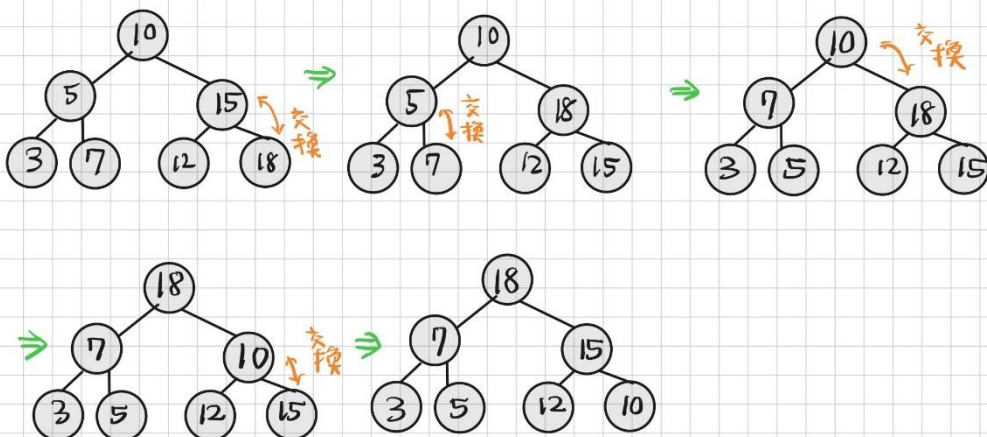
input 1 = 10, 5, 15, 3, 7, 12, 18

Min Heap =



input 1 = 10, 5, 15, 3, 7, 12, 18

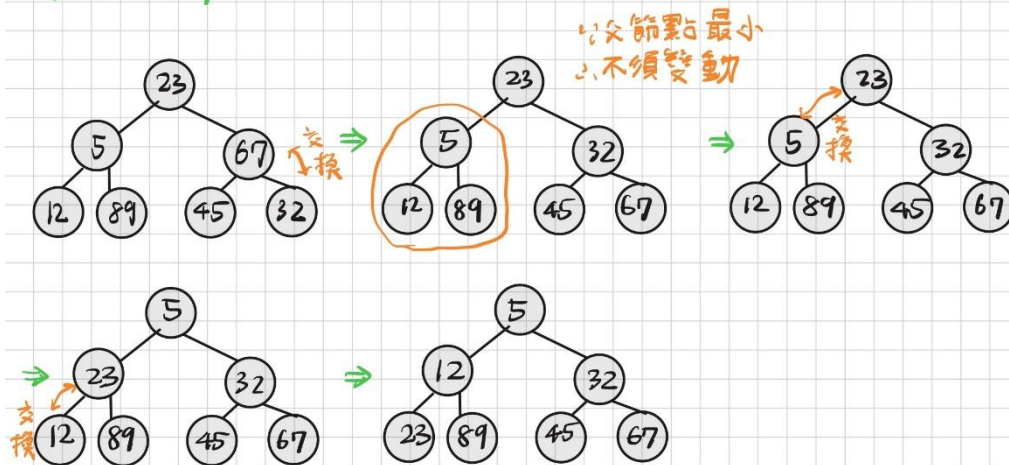
Max Heap =



Input 2: 23, 5, 67, 12, 89, 45, 32

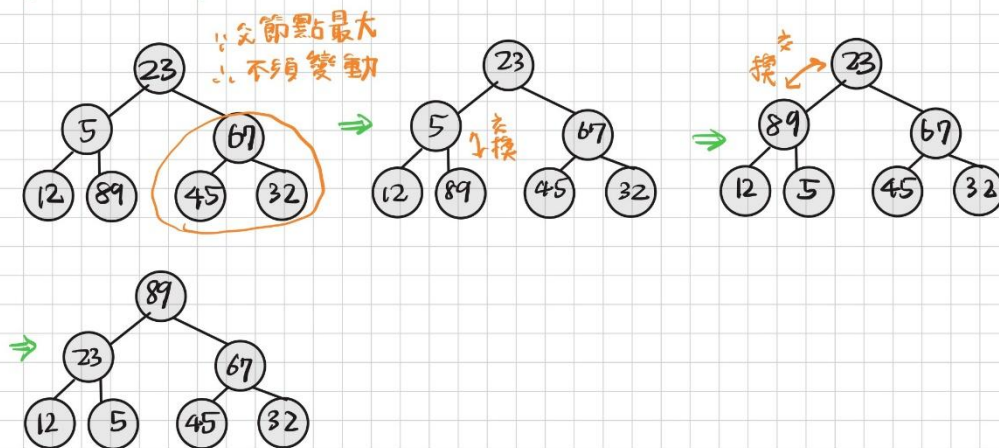
input 2 = 23, 5, 67, 12, 89, 45, 32

Min Heap =



input 2 = 23, 5, 67, 12, 89, 45, 32

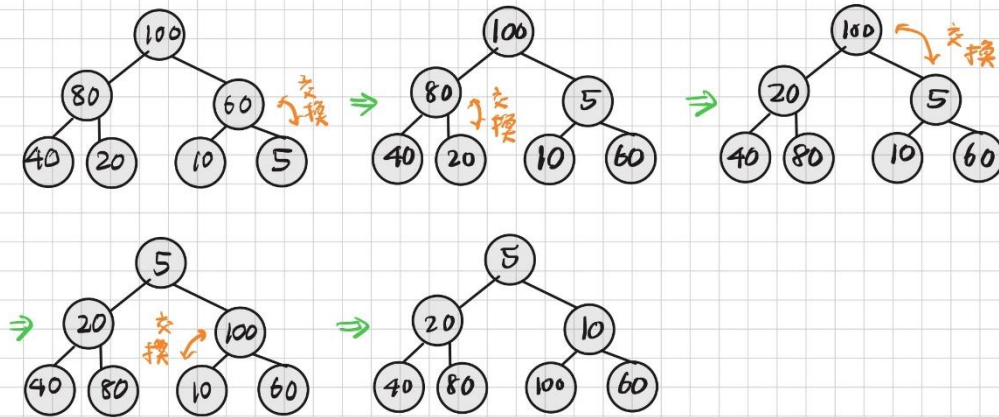
Max Heap =



Input 3: 100,80,60,40,20,10,5

input 3: 100, 80, 60, 40, 20, 10, 5

Min Heap =

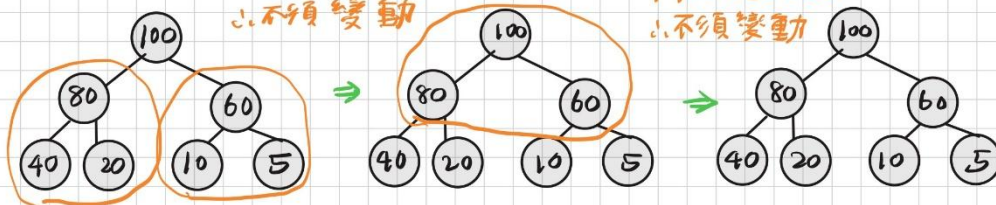


input 3: 100, 80, 60, 40, 20, 10, 5

Max Heap =

∵ 80, 60 父節點皆為最大
∴ 不須變動

∵ 100 > 80 > 60
∴ 不須變動



Lab10-Q2-1

這題要求實作 Max Heap 程式。首先先隨機讀取同資料夾三個 txt 檔案中的其中一個，接著再實作 Max Heap 程式，將輸入的數列轉換成 Max Heap 並輸出。

Code

```
// Max Heap 程式
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
#include <random> // 引入隨機數標頭檔
#include <ctime>  // 引入時間標頭檔用以初始化隨機種子
using namespace std;

// 從文件中讀取數據並存入向量
vector<int> readFromFile(const string& filename) {
    vector<int> arr;
    ifstream file(filename);

    if (!file) {
        cerr << "Error opening file: " << filename << endl;
        return arr;
    }

    string line;
    while (getline(file, line)) { // 持續讀取整行內容
        stringstream ss(line);    // 創建字符串流
        string value;
        while (getline(ss, value, ',')) { // 用逗號分隔值
            try {
                arr.push_back(stoi(value)); // 將字符串轉換為整數並存入向量
            }
            catch (exception& e) {
                cerr << "Invalid number format in file: " << value << endl;
            }
        }
    }
}
```

```

file.close();
return arr;
}

class MaxHeap {
public:
    vector<int> heap; // 儲存 Max Heap 的元素

    // 建立 Max Heap
    void buildMaxHeap(vector<int>& arr) { // 建立 Max Heap
        heap = arr;

        for (int i = (heap.size() / 2) - 1; i >= 0; i--) { // 從最後一個非葉子節點開始向上執行 Max Heap
            heapify(i);
        }
    }

    void heapify(int i) { // 堆化函式 (確保以 i 為根的子樹符合 Max Heap 性質)
        int largest = i; // 假設當前節點是最大的
        int left = 2 * i + 1; // 左子節點索引
        int right = 2 * i + 2; // 右子節點索引

        // if 判斷式，檢查左子節點是否為有效範圍且比當前節點(父節點)大
        // 如果判斷式成立，把 largest 設為 left
        // 第一步，檢查左子節點
        // left < heap.size(), 透過邊界檢查，來確保計算出來的左子節點索引沒有超出陣列的有效範圍
        // heap[left] > heap[largest], 比較左子節點的值是否大於目前記錄的最大值(父節點)
        // 若兩者皆成立，代表左子節點大於父節點，違反 Max Heap 性質
        if (left < heap.size() && heap[left] > heap[largest]) { //
            largest = left; // 將最大值指標 largest 轉向左子節點的索引位置
        }

        // if 判斷式，檢查右子節點是否為有效範圍且比當前節點(父節點)大
        // 如果判斷式成立，把 largest 設為 right
        // 第二步，檢查右子節點

```

// right < heap.size(), 透過邊界檢查, 來確保計算出來的右子節點索引沒有超出陣列的有效範圍

// heap[right] > heap[largest], 比較右子節點的值是否比目前的最大值位置還要大

// 若兩者皆成立, 代表右子節點是三者之中的最大值

```
if (right < heap.size() && heap[right] > heap[largest]) {  
    largest = right; // 將最大值指標 largest 更新為右子節點的索引位置  
}
```

// if 判斷式, 如果最大的不是父節點, 交換並繼續堆化

// 遞迴處理受影響的子樹

// 第三步: 若最大值不是父節點, 執行交換與遞迴調整

// 若 largest 不等於 i, 則代表經過上面兩個 if 的比較後, 左子或右子節點的值比父節點還要大

```
if (largest != i) {  
    swap(heap[i], heap[largest]); // 為了維持 Max Heap 原則, 將父節點的值與最大子節點的值進行實體位置對調
```

heapify(largest); // 因為原本較小的父節點被換下去, 可能會導致該位置以下的子樹結構被破壞

// 因此, 必須以剛才被換下去的新位置 (即 largest 索引) 作為新的根節點

// 再次遞迴呼叫

heapify, 像漣漪一樣持續向下修正, 直到整棵子樹都安全地符合 Max Heap 為止

```
}  
}
```

// 顯示 Heap 的內容 (使用 BFS)

```
void printHeap() {
```

```
    vector<int> backup = heap;
```

```
    while (!heap.empty()) {
```

```
        cout << heap[0] << " "; // 列印出樹頂 heap[0] (當前最大值)
```

```
        if (heap.size() == 1) {
```

```
            heap.pop_back();
```

```
            break;
```

```
        }
```

```
        heap[0] = heap.back();
```

```

    heap.pop_back();

    heapify(0);
}

heap = backup;
cout << endl;
}
};

int main() {
    // 將三個 input.txt 檔案名稱存入向量中
    vector<string> files = {"input1.txt", "input2.txt", "input3.txt"};

    // 設定隨機數生成器
    // 使用 random_device 與 mt19937 虛擬隨機數生成引擎，並用目前時間 (time(0)) 當作隨機種子
    mt19937 gen(static_cast<unsigned int>(time(0)));
    uniform_int_distribution<> dis(0, files.size() - 1); // 產生 0 到 2 的均勻分佈隨機數

    // 隨機選擇一個索引並選出檔案
    int randomIndex = dis(gen);
    string filename = files[randomIndex]; // 隨機挑選出的檔案名稱

    vector<int> arr = readFromFile(filename); // 讀取數據

    if (arr.empty()) { // 如果數據為空
        cerr << "No valid data found in file." << endl; // 輸出錯誤信息
        return -1;
    }

    cout << "Input Array: "; // 輸出讀取的數據
    for (int val : arr) {
        cout << val << " "; // 輸出每個元素
    }
    cout << endl;
}

```



```

MaxHeap maxHeap; // 創建 Max Heap 對象
maxHeap.buildMaxHeap(arr); // 建立 Max Heap

// 輸出 Max Heap 的內容
cout << "Max Heap: ";
maxHeap.printHeap();
cout << endl;

system("pause");
return 0;
}

```

Result

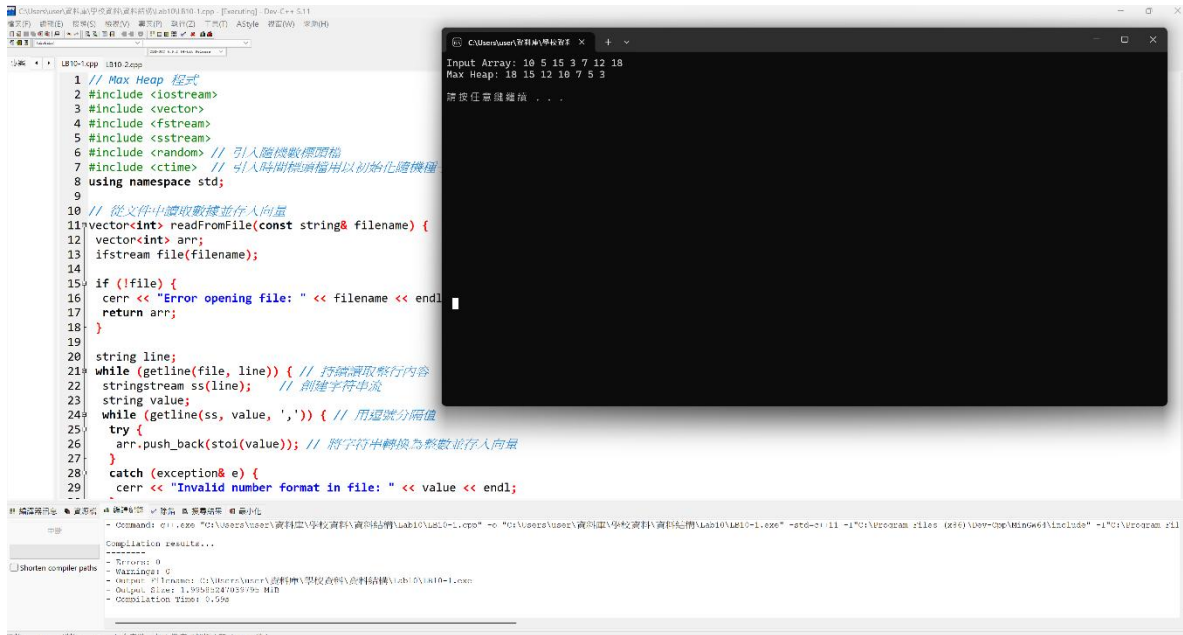
Input1.txt: 10,5,15,3,7,12,18



```

C:\Users\user\資料庫\學校資料 >
Input Array: 10 5 15 3 7 12 18
Max Heap: 18 15 12 10 7 5 3
請按任意鍵繼續 . . . |

```



```

1 // Max Heap 程式
2 #include <iostream>
3 #include <vector>
4 #include <fstream>
5 #include <sstream>
6 #include <random> // 引入隨機數標頭檔
7 #include <ctime> // 引入時間標頭檔用以初始化隨機種子
8 using namespace std;
9
10 // 從文件中讀取數據並存入向量
11 vector<int> readFromFile(const string& filename) {
12     vector<int> arr;
13     ifstream file(filename);
14
15     if (!file) {
16         cerr << "Error opening file: " << filename << endl;
17         return arr;
18     }
19
20     string line;
21     while (getline(file, line)) { // 持續讀取整行內容
22         stringstream ss(line); // 創建字串流
23         string value;
24         while (getline(ss, value, ',')) { // 用逗號分隔值
25             try {
26                 arr.push_back(stoi(value)); // 將字符串轉換為整數並存入向量
27             }
28             catch (exception& e) {
29                 cerr << "Invalid number format in file: " << value << endl;
30             }
31         }
32     }
33     return arr;
34 }
35
36 int main() {
37     vector<int> arr = readFromFile("Input1.txt");
38     MaxHeap maxHeap;
39     maxHeap.buildMaxHeap(arr);
40     maxHeap.printHeap();
41     system("pause");
42     return 0;
43 }

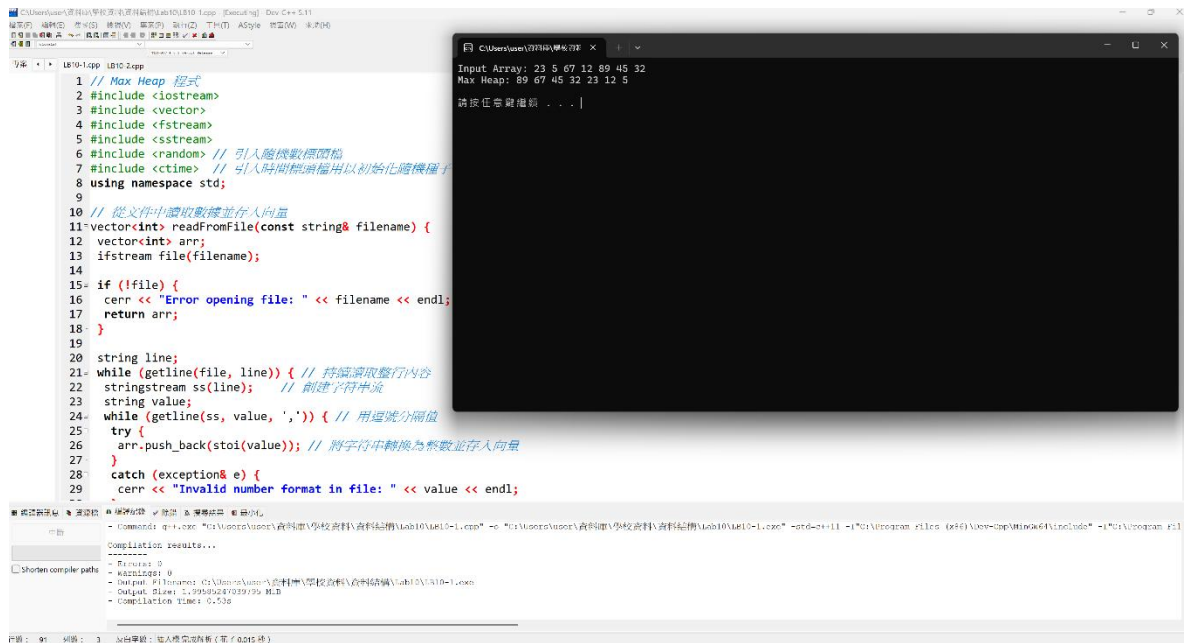
```

Input2.txt: 23,5,67,12,89,45,32

```
C:\Users\user\資料庫\學校資料\Lab10\Lab10-1.cpp [Executing] Dev-C++ 5.11

Input Array: 23 5 67 12 89 45 32
Max Heap: 89 67 45 32 23 12 5

請按任意鍵繼續 . . . |
```



Input3.txt: 100,80,60,40,20,10,5

```
C:\Users\User\資料庫\學校資料 > + v
Input Array: 100 80 60 40 20 10 5
Max Heap: 100 80 60 40 20 10 5

請按任意鍵繼續 . . . |
```

```
1 // Max Heap 程式
2 #include <iostream>
3 #include <vector>
4 #include <fstream>
5 #include <sstream>
6 #include <random> // 引入隨機數生成函數
7 #include <ctime> // 引入時間標頭以初始化隨機種子
8 using namespace std;
9
10 // 從文件中讀取數據並存入向量
11 vector<int> readFromFile(const string& filename) {
12     vector<int> arr;
13     ifstream file(filename);
14
15     if (!file) {
16         cerr << "Error opening file: " << filename << endl;
17         return arr;
18     }
19
20     string line;
21     while (getline(file, line)) { // 持續讀取整行內容
22         stringstream ss(line); // 創建字符串流
23         string value;
24         while (getline(ss, value, ',')) { // 用逗號分割值
25             try {
26                 arr.push_back(stoi(value)); // 將字符串轉換為整數並存入向量
27             }
28             catch (exception& e) {
29                 cerr << "Invalid number format in file: " << value << endl;
30             }
31         }
32     }
33     return arr;
34 }
```

Output window:

```
Input Array: 100 80 60 40 20 10 5
Max Heap: 100 80 60 40 20 10 5

請按任意鍵繼續 . . . |
```

Compilation results:

```
g++ -std=c++11 -c "C:\Users\User\資料庫\學校資料\Lab10\Lab10-1.cpp" -o "C:\Users\User\資料庫\學校資料\Lab10\Lab10-1.o"
g++ -std=c++11 -o "C:\Users\User\資料庫\學校資料\Lab10\Lab10-1.exe" "C:\Users\User\資料庫\學校資料\Lab10\Lab10-1.o"
Output filename: C:\Users\User\資料庫\學校資料\Lab10\Lab10-1.exe
Output Size: 149520 bytes (14.5 KB)
Compilation Time: 0.55s
```

Lab10-Q2-2

這題要求實作 Min Heap 程式。首先先隨機讀取同資料夾三個 txt 檔案中的其中一個，接著再實作 Min Heap 程式，將輸入的數列轉換成 Min Heap 並輸出。

Code

```
// Min Heap 程式
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
#include <random> // 引入隨機數標頭檔
#include <ctime>  // 引入時間標頭檔以便初始化隨機種子
using namespace std;

// 從文件中讀取數據並存入向量
vector<int> readFromFile(const string& filename) {
    vector<int> arr;
    ifstream file(filename);

    if (!file) {
        cerr << "Error opening file: " << filename << endl;
        return arr;
    }

    string line;
    while (getline(file, line)) { // 持續讀取整行內容
        stringstream ss(line);    // 創建字符串流
        string value;
        while (getline(ss, value, ',')) { // 用逗號分隔值
            try {
                arr.push_back(stoi(value)); // 將字符串轉換為整數並存入向量
            }
            catch (exception& e) {
                cerr << "Invalid number format in file: " << value << endl;
            }
        }
    }
}
```

```

file.close();
return arr;
}

class MinHeap {
public:
    vector<int> heap; // 儲存 Min Heap 的元素

    // 建立 Min Heap
    void buildMinHeap(vector<int>& arr) { // 建立 Min Heap
        heap = arr;

        for (int i = (heap.size() / 2) - 1; i >= 0; i--) { // 從最後一個非葉子節點開始向上執行 Min Heap
            heapify(i);
        }
    }

    void heapify(int i) { // 堆化函式 (確保以 i 為根的子樹符合 Min Heap 性質)
        int smallest = i; // 假設當前節點是最小的
        int left = 2 * i + 1; // 左子節點索引
        int right = 2 * i + 2; // 右子節點索引

        // if 判斷式，檢查左子節點是否為有效範圍且比當前節點(父節點)小
        // 如果判斷式成立，把 smallest 設為 left
        // 第一步，檢查左子節點
        // left < heap.size(), 透過邊界檢查，來確保計算出來的左子節點索引沒有超出陣列的有效範圍
        // heap[left] < heap[smallest], 比較左子節點的值是否小於目前記錄的最小值(父節點)
        // 若兩者皆成立，代表左子節點小於父節點，違反 Min Heap 性質
        if (left < heap.size() && heap[left] < heap[smallest]) {
            smallest = left; // 將最小值指標 smallest 轉向左子節點的索引位置
        }

        // if 判斷式，檢查右子節點是否為有效範圍且比當前節點(父節點)小
        // 如果判斷式成立，把 smallest 設為 right
        // 第二步，檢查右子節點

```

```

// right < heap.size(), 透過邊界檢查, 來確保計算出來的右子節點索引沒有超出陣列的有效範圍
// heap[right] < heap[smallest], 比較右子節點的值是否小於目前為止的最小值
// 若兩者皆成立, 代表右子節點是三者之中的最小值
if (right < heap.size() && heap[right] < heap[smallest]) {
    smallest = right; // 將最小值指標 smallest 更新為右子節點的索引位置
}

// if 判斷式, 如果最小的不是父節點, 交換並繼續堆化
// 遞迴處理受影響的子樹
// 第三步: 若最小值不是父節點, 執行交換與遞迴調整
// 若 smallest 不等於 i, 則代表經過上面兩個 if 的比較後, 左子或右子節點的值比父節點還要小
if (smallest != i) {
    swap(heap[i], heap[smallest]); // 為了維持 Min Heap 原則, 將父節點的值與最小子節點的值進行實體位置對調
    heapify(smallest); // 因為原本較大的父節點被換下去, 可能會導致該位置以下的子樹結構被破壞
    // 因此, 必須以剛才被換下去的新位置 (即 smallest 索引) 作為新的根節點
    // 再次遞迴呼叫
}
heapify, 像漣漪一樣持續向下修正, 直到整棵子樹都安全地符合 Min Heap 為止
}
}

// 顯示 Heap 的內容 (使用 BFS)
void printHeap() {
    vector<int> backup = heap;

    while (!heap.empty()) {
        cout << heap[0] << " "; // 列印出樹頂 heap[0] (當前最小值)

        if (heap.size() == 1) {
            heap.pop_back();
            break;
        }

        heap[0] = heap.back();
    }
}

```

```

    heap.pop_back();

    heapify(0);
}

heap = backup;
cout << endl;
}
};

int main() {
// 將三個 input.txt 檔案名稱存入向量中
vector<string> files = {"input1.txt", "input2.txt", "input3.txt"};

// 設定隨機數生成器
// 使用 random_device 與 mt19937 虛擬隨機數生成引擎，並用目前時間 (time(0)) 當作隨機種子
mt19937 gen(static_cast<unsigned int>(time(0)));
uniform_int_distribution<> dis(0, files.size() - 1); // 產生 0 到 2 的均勻分佈隨機數

// 隨機選擇一個索引並選出檔案
int randomIndex = dis(gen);
string filename = files[randomIndex]; // 隨機挑選出的檔案名稱

vector<int> arr = readFromFile(filename); // 讀取數據

if (arr.empty()) { // 如果數據為空
    cerr << "No valid data found in file." << endl; // 輸出錯誤信息
    return -1;
}
cout << "Input Array: "; // 輸出讀取的數據
for (int val : arr) {
    cout << val << " "; // 輸出每個元素
}
cout << endl;

MinHeap minHeap; // 創建 Min Heap 對象

```

```

minHeap.buildMinHeap(arr); // 建立 Min Heap

// 輸出 Min Heap 的內容
cout << "Min Heap: ";
minHeap.printHeap();
cout << endl;

system("pause");
return 0;
}

```

Result

Input1.txt: 10,5,15,3,7,12,18

```

C:\Users\user\資料庫\學校資料 >
Input Array: 10 5 15 3 7 12 18
Min Heap: 3 5 7 10 12 15 18

請按任意鍵繼續 . . . |

```

```

1 // Min Heap 程式
2 #include <iostream>
3 #include <vector>
4 #include <fstream>
5 #include <sstream>
6 #include <random> // 引入隨機數標頭檔
7 #include <ctime> // 引入時間標頭檔以便初始化隨機種子
8 using namespace std;
9
10 // 從文件中讀取數據並存入向量
11 vector<int> readFromFile(const string& filename) {
12     vector<int> arr;
13     ifstream file(filename);
14
15     if (!file) {
16         cerr << "Error opening file: " << filename << endl;
17         return arr;
18     }
19
20     string line;
21     while (getline(file, line)) { // 持續讀取整行內容
22         stringstream ss(line); // 創建字串流
23         string value;
24         while (getline(ss, value, ',')) { // 用逗號分隔值
25             try {
26                 arr.push_back(stoi(value)); // 將字串轉換為整數並存入向量
27             }
28             catch (exception& e) {
29                 cerr << "Invalid number format in file: " << value << endl;
30             }
31         }
32     }
33     return arr;
34 }
35
36 int main() {
37     string filename = "Input1.txt";
38     vector<int> arr = readFromFile(filename);
39
40     if (arr.empty()) {
41         cerr << "No data found in the input file." << endl;
42         return 1;
43     }
44
45     minHeap minHeap(arr);
46     minHeap.buildMinHeap(arr);
47     minHeap.printHeap();
48     system("pause");
49     return 0;
50 }

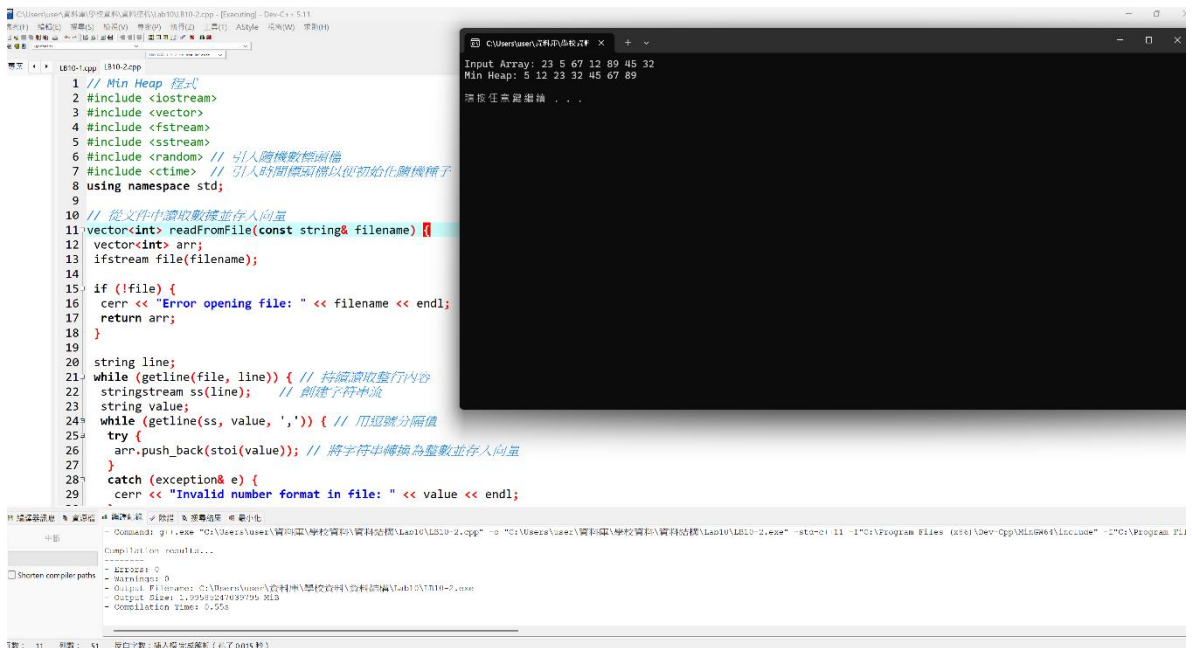
```

```

Command: g++ "C:\Users\user\資料庫\學校資料\LB10-2.cpp" -o "C:\Users\user\資料庫\學校資料\LB10-2.exe" -std=c++11 -I"C:\Program Files (x86)\Dev-Cpp\MinGW64\include" -I"C:\Program Files (x86)\Dev-Cpp\MinGW64\lib"
Compilation results...
-----
- No warnings
- Warnings: 0
- Output: C:\Users\user\資料庫\學校資料\LB10-2.exe
- Output: g++ -std=c++11 -I"C:\Program Files (x86)\Dev-Cpp\MinGW64\include" -I"C:\Program Files (x86)\Dev-Cpp\MinGW64\lib" "C:\Users\user\資料庫\學校資料\LB10-2.cpp" -o "C:\Users\user\資料庫\學校資料\LB10-2.exe"
- Compilation Time: 0.03s

```


Input2.txt: 23,5,67,12,89,45,32



Input3.txt: 100,80,60,40,20,10,5

```
C:\Users\user\資料庫\學校資料 × + ▾
Input Array: 100 80 60 40 20 10 5
Min Heap: 5 10 20 40 60 80 100

請按任意鍵繼續 . . .
```

The image displays a C++ development environment with a code editor and a terminal window.

Code Editor:

```
// Min Heap 程式
1 //include <iostream>
2 #include <vector>
3 #include <fstream>
4 #include <sstream>
5 #include <random> // 引入隨機數相關函數
6 #include <ctime> // 引入時間相關函數以便初始化隨機種子
7 using namespace std;
8
9 // 從文件中讀取數據並存入向量
10 vector<int> readFromFile(const string& filename) {
11     vector<int> arr;
12     ifstream file(filename);
13
14     if (!file) {
15         cerr << "Error opening file: " << filename << endl;
16         return arr;
17     }
18
19     string line;
20     while (getline(file, line)) { // 持續讀取整行內容
21         stringstream ss(line); // 創建字符串流
22         string value;
23         while (getline(ss, value, ',')) { // 用逗號分隔值
24             try {
25                 arr.push_back(stoi(value)); // 將字符串轉換為整數並存入向量
26             }
27             catch (exception& e) {
28                 cerr << "Invalid number format in file: " << value << endl;
29             }
30         }
31     }
32 }
```

Terminal Window:

```
C:\Users\user> g++ 10-2.cpp -std=c++11
Input Array: 100 80 60 40 20 10 5
Min Heap: 5 10 20 40 60 80 100
請按任意鍵繼續 . . .
```

Output Window:

```
4 編譯器 5 輸出 6 錯誤 7 警告 8 信息 9 其他
Compilation results...
- Frontend: g++
- Backend: g++
- Output filename: C:\Users\user\Documents\學校資料\學校資料結構\Lab10\10-2.exe
- Output files: 1,490,857,387,515,816
- Compilation time: 0.22s
```